

Appunti S2 — Autenticazione

Appunti — Modulo S2: Autenticazione

Corso: Lab Sicurezza Informatica T **Materiale di riferimento:**
lezione_moduloS2_autenticazione.md **Appunti grezzi:** APPUNTI GREZZI/
Lab - Security/Appunti_grezzi_lezioneS2.md **Elaborati:** 2026-06-23

AAA — Autenticazione, Autorizzazione, Auditing

Autenticazione è l'attribuzione certa dell'identità di un soggetto che usa il sistema. Include un'**identificazione** preliminare, che però non è sufficiente a dichiararsi autenticati. L'identificazione è il **nome che dichiarare**, mentre l'autenticazione è la **prova che quel nome sia davvero tuo**.

Autorizzazione è la verifica dei **diritti** di un soggetto di **compiere un'azione su un determinato oggetto** nel sistema: o si ha il permesso o non lo si ha. I propri permessi vengono verificati **dopo l'autenticazione**. Su Linux, **PAM** gestisce l'autenticazione, **sudo** e i **permessi su file** gestiscono l'autorizzazione.

Auditing è il tracciamento delle **decisioni di autenticazione e autorizzazione**. È la base di un compromesso: più dettagliato è il log, più ne risentono prestazioni e privacy.

✓ Distinzione AAA ottimamente formulata. La precisione su "identificazione ≠ autenticazione" è esattamente il livello che il quiz teorico testa. Anche il collegamento PAM/sudo per Linux è corretto.

I quattro fattori dell'autenticazione

L'autenticazione si basa su ciò che l'utente **conosce, possiede, è, o fa**. Il **Prover (P)** deve dimostrare al **Verifier (V)** di conoscere un segreto di uno dei tipi seguenti.

Fattore	Esempi	Punto di forza	Punto di debolezza
Conosce	password, PIN, risposte segrete	Nessun hardware richiesto	Se fuoriesce dal canale o dal DB di V, è bruciato
Possiede	hard-token, Yubikey, chiavetta NFC	Non si può clonare via rete	Può essere smarrito o rubato fisicamente
È (biometrico)	iride, impronta digitale	Unico e sempre presente	Irrevocabile se compromesso — non si cambia come una password
Fa / Dove è	GPS, geolocalizzazione	Segnale di anomalia contestuale	Non sufficiente da solo come fattore di accesso

✓ Corretto anche il punto sul GPS: non si fa login con la posizione, ma si usa come segnale di allerta per forzare un secondo step.

/etc/shadow: dove vive il segreto lato server

/etc/shadow contiene la **fingerprint della password**, una versione hashed ottenuta dalla **concatenazione password || salt**:

```
$6$ViDM2ItuaSNPBxf0$Ip40Uoaqu0.0iFafaIVeMazZpIisBV.CC76...
```

- \$6\$ = identificatore algoritmo hash (SHA-512)
- ViDM2ItuaSNPBxf0 = **salt** (variazione random generata al momento della scelta della password)
- il resto = **fingerprint** (hash della concatenazione)

Il **salt** rompe la simmetria: due utenti con la stessa password hanno fingerprint diverse. Impedisce le **rainbow tables** (tabelle precalcolate di hash), perché bisognerebbe precalcolarle per ogni possibile salt.

Il **salt non basta contro attacchi offline**: chi ruba `/etc/shadow` può tentare brute force sul proprio hardware senza limiti di tentativi. La contromisura principale è avere una **password con alta entropia**.

Il **pepper** è una variante del salt che vive in un **HSM (Hardware Security Module)**, non nel database: anche rubando `/etc/shadow`, l'attaccante non ha il pepper e non può replicare il calcolo del fingerprint.

Domanda: cos'è un HSM nel pratico? che forma ha?

Un HSM è un dispositivo hardware dedicato **esclusivamente** a operazioni crittografiche: generare chiavi, firmare, cifrare, calcolare hash con segreti interni. Il suo elemento distintivo è che le chiavi e i segreti che contiene **non ne escono mai in chiaro** — anche se qualcuno ha accesso fisico al dispositivo, questo si autodistrugge (zeroing della memoria) se viene manomesso.

Forme fisiche: in contesti enterprise ha l'aspetto di un rack server 1U (es. Thales Luna, AWS CloudHSM). In contesti più piccoli può essere una smart card o un modulo USB. Sul banco del professore potrebbe non mostrartene uno fisicamente — ma concettualmente è un “caveau crittografico”: il tuo `/etc/shadow` contiene `hash(password || pepper)`, ma il pepper non è mai scritto sul disco, vive solo nell'HSM. Un attaccante che ruba il disco non può calcolare il fingerprint perché non conosce il pepper. Anche il sysadmin che gestisce il sistema non vede il pepper in chiaro — solo l'HSM lo conosce.

Scelta della password: l'entropia conta più della complessità percepita

Domanda: cos'è l'entropia? in base a cosa si ottiene? cosa la determina?

Entropia in crittografia misura quanto è difficile indovinare una password per tentativi. Si misura in **bit**: N bit di entropia significa che un attaccante deve fare in media 2^N tentativi per trovarla.

Due fattori la determinano: 1. **Dimensione del pool**: quanti simboli diversi uso (solo minuscole = 26; maiuscole+minuscole+numeri = 62; tutti i caratteri ASCII stampabili = 95) 2. **Lunghezza**: quanti simboli ho nella password

Formula: $\text{entropia} = \text{lunghezza} \times \log_2(\text{dimensione_pool})$

Quindi: 8 caratteri da 95 possibili $\rightarrow 8 \times 6.57 \approx 52$ **bit teorici**.

Il punto chiave: quei 52 bit valgono solo se la scelta è **veramente casuale**. Se invece uso un pattern prevedibile (una parola base + sostituzione di lettere: Tr0ub4dor&3), l'attaccante non cerca tra tutti i $95^8 = 10^{15}$ simboli — cerca tra le parole comuni con varianti note. L'entropia reale scende a ~28 bit.

correct horse battery staple ha ~44 bit di entropia **se** le parole sono scelte a caso da un dizionario grande — anche se sembra semplice, la combinazione casuale di 4 parole da un vocabolario di 2048 parole dà $2048^4 = 2^{44}$ possibilità, tutte equivalenti. È più difficile da bruteforare e più facile da ricordare.

Perché gli LLM non generano buona entropia: tendono a produrre output statisticamente prevedibili (parole comuni, strutture tipiche). Una password “casuale” generata da un LLM non è uniformemente distribuita nello spazio delle possibilità — un attaccante che sa che è stata generata da un LLM può restringere lo spazio di ricerca.

Soluzione pratica: un **password manager** genera password con vera casualità computazionale (CSPRNG) e le memorizza cifrate. Una sola passphrase forte sblocca il vault.

Autenticazione passiva vs attiva

L'**autenticazione passiva** è il modello della password classica: P e V concordano un segreto, P lo invia a V che lo confronta con la propria copia. Ha problemi strutturali: se il segreto viaggia in chiaro viene intercettato; se è offuscato ma sempre uguale, basta catturarlo e riprodurlo (*replay attack*). E V possiede una copia del segreto: se V viene compromesso, tutti i segreti sono a rischio.

Domanda: spiega più chiaramente l'autenticazione attiva — pensavo fosse una sorta di 2FA ma non lo è.

Hai ragione che non è 2FA. La distinzione è un'altra: **non cambia il numero di fattori, cambia cosa viene inviato sul canale.**

- **Passiva:** P invia il segreto stesso → V confronta. Se qualcuno intercetta: ha il segreto.
- **Attiva:** P invia una *prova che conosce il segreto* senza inviare il segreto. E questa prova è diversa ogni volta → se qualcuno intercetta: ha solo una prova già consumata, inutilizzabile.

Analogia: immagina di dover dimostrare di conoscere una passphrase segreta con un giudice. - **Passiva:** la sussurri all'orecchio del giudice. Se qualcuno ascolta, ha la passphrase. - **Attiva:** il giudice ti lancia un dado e ti dice “dimmi il terzo carattere della passphrase + il numero sul dado”. Dimostri di conoscerla senza rivelarla tutta, e la risposta è diversa ogni volta.

Il rischio che resta nell'attiva è il **MITM attivo**: se un attaccante si frappone tra P e V, può impersonare V verso P e usare le risposte di P per autenticarsi verso V. La cifratura del canale (TLS) risolve questo.

S-KEY One-Time Password: hash come ratchet

S-KEY è autenticazione attiva basata su catena di hash. Concetto chiave: **applicare l'hash è facile; invertirlo è impossibile.**

Setup: P conosce un segreto N. V viene inizializzato con $h^k(N)$ (hash applicato k volte a N).

Prima autenticazione: P invia $h^{(k-1)}(N)$. V verifica: calcola $\text{hash}(h^{(k-1)}(N))$ e controlla che sia uguale a $h^k(N)$. Se sì, accetta. Poi V aggiorna il suo riferimento a $h^{(k-1)}(N)$.

Seconda autenticazione: P invia $h^{(k-2)}(N)$. V applica hash una volta e controlla. E così via.

Domanda (implicita — “non ho capito l'argomento” Q5): perché chi intercetta non può riusare il token?

Se un attaccante intercetta $h^{(k-1)}(N)$ sul canale, ha in mano esattamente quello che P ha appena inviato. Ma V ha già aggiornato il suo riferimento a $h^{(k-1)}(N)$ — quel token è consumato, non vale più.

Per la prossima autenticazione serve $h^{(k-2)}(N)$. L'attaccante dovrebbe invertire l'hash per ricavarlo da $h^{(k-1)}(N)$ — ma l'hash è a senso unico: non è calcolabile all'indietro. Il ratchet va solo in avanti.

È lo stesso motivo per cui il sale in `/etc/shadow` non si può invertire: hash è facile in un senso, impossibile nell'altro.

Sfida e risposta: SSH e la chiave privata come segreto

⚠ Questa sezione non era presente negli appunti grezzi.

Il meccanismo challenge-response con crittografia asimmetrica è quello che usi ogni volta che ti connetti via SSH con chiave pubblica.

Setup: P genera una coppia chiave (PRIV_P, PUB_P). Mette PUB_P in `~/.ssh/authorized_keys` sul server (V).

Flusso di autenticazione: 1. V genera un numero casuale (nonce) e lo cifra con PUB_P → invia la sfida cifrata 2. P decifra con PRIV_P — può farlo solo chi possiede la chiave privata 3. P rimanda il nonce a V 4. V confronta: se coincide, P ha dimostrato di possedere PRIV_P senza averla mai inviata

Perché è più robusta della password: anche intercettando tutto il traffico, l'attaccante vede solo il nonce cifrato e la risposta — nulla di riutilizzabile. L'unico rischio è la compromissione di PRIV_P sul dispositivo di P.

Connessione SysAdmin: il file `~/.ssh/authorized_keys` che hai già configurato in SysAdmin è esattamente questo: V che memorizza PUB_P per il confronto.

/etc/pam.d/: il framework che collega tutto a Linux

PAM (**Pluggable Authentication Modules**) è il livello che ogni programma Linux usa per delegare l'autenticazione — login, ssh, sudo, su non implementano la logica da soli: la passano a PAM, che carica moduli configurabili. La configurazione vive in /etc/pam.d/, un file per programma.

Domanda: cos'è PAM? tabella chiara dei parametri, specialmente control-flag.

Formato di ogni riga in /etc/pam.d/<programma>:

module-type control-flag module-path [argomenti]

Tabella: module-type

Tipo	Funzione
auth	Verifica dell'identità: confronta credenziali (es. legge /etc/shadow)
account	Controlli non legati alla password: account scaduto? restrizioni orarie? gruppo corretto?
session	Setup e teardown della sessione: mount home, variabili d'ambiente, logging
password	Aggiornamento delle credenziali: robustezza nuova password, scadenza

Tabella: control-flag — il cuore della logica di stack

Flag	Cosa succede al fallimento	Cosa succede al successo	Continua?
required	Segna stack come fallito, ma continua verso i moduli successivi	Contribuisce al successo	Sempre

Flag	Cosa succede al fallimento	Cosa succede al successo	Continua?
	(nasconde quale modulo ha fallito)		
requisite	Ferma immediatamente lo stack con fallimento	Contribuisce al successo	No (solo se fallisce)
sufficient	Ignorato	Ferma immediatamente lo stack con successo (purché nessun required precedente abbia già fallito)	No (solo se riesce)
optional	Ignorato (a meno che sia l'unico modulo presente)	Ignorato	Sempre

Domanda: perché l'ordine dei moduli è importante?

Perché i flag si combinano in sequenza, e lo stesso flag ha effetto diverso in base a cosa è già successo prima.

Esempio:

```
auth    required    pam_unix.so
auth    sufficient  pam_krb5.so
```

Qui `pam_unix` (password Unix) deve passare per forza (`required`). Se passa e `pam_krb5` (Kerberos) fallisce: lo stack riesce comunque — il `sufficient` di `pam_krb5` non interrompe negativamente, e `pam_unix` era già `required` e passato.

Se inverti l'ordine:

```
auth    sufficient  pam_krb5.so
auth    required    pam_unix.so
```

Ora: se `pam_krb5` riesce, lo stack si ferma subito con successo — `pam_unix` non viene nemmeno consultato. L'utente può autenticarsi con Kerberos senza dover avere la password Unix.

Stessa configurazione, ordine diverso → logica completamente diversa. L'ordine non è decorativo.

Moduli PAM comuni

Modulo	Tipo	Funzione
pam_unix.so	auth/password	Autenticazione Unix classica via /etc/shadow
pam_tally.so	auth	Conta tentativi falliti, blocca account dopo N → difesa brute force online
pam_cracklib.so	password	Verifica robustezza nuova password durante il cambio
pam_nologin.so	auth	Se /etc/nologin esiste, solo root accede (manutenzione)
pam_deny.so	tutti	Sempre fallisce — usato in /etc/pam.d/other per default-deny
pam_warn.so	tutti	Logga il tentativo — usato insieme a pam_deny.so

Esempio default-deny (/etc/pam.d/other): qualsiasi programma senza una configurazione PAM dedicata viene bloccato da pam_warn (loga) + pam_deny (rifiuta). La sicurezza per fail-closed: se non è esplicitamente permesso, è vietato.

2FA vs 2SA vs MFA

2FA (Two-Factor Authentication): richiede 2 fattori che siano **categorialmente distinti** — conosce + possiede, oppure possiede + è. La distinzione categoriale è il requisito chiave.

2SA (Two-Step Authentication): richiede 2 passi ma non necessariamente due fattori distinti. OTP via SMS o email sono 2SA: il canale SMS/email è intercettabile (MITM), quindi non costituisce un “possesso” affidabile. Chi controlla il canale può intercettare il codice.

Caso limite: sbloccare un'app con PIN per ottenere il token OTP è “conoscenza aggiuntiva”, non un secondo fattore distinto — sia la password che il PIN rientrano nella categoria “conosce”.

MFA (Multi-Factor Authentication): generalizza 2FA a più di due fattori categorialmente distinti.

Perché importa per l'esame: 2SA protegge contro chi ha solo la password rubata, non contro chi controlla il canale. 2FA con hardware dedicato protegge anche contro il controllo del canale.

✓ Distinzione 2FA/2SA/MFA ben catturata e con la motivazione corretta (intercettabilità del canale SMS).

FIDO: standard per autenticazione forte su larga scala

FIDO (**Fast IDentity Online**) è un consorzio (Google, Microsoft, Yubico e altri) che ha sviluppato standard per rendere l'**autenticazione forte** usabile su larga scala.

⚠ **Correzione:** negli appunti grezzi FIDO viene descritto come “standard per rendere l'autorizzazione a doppia chiave usabile”. È un errore di terminologia: FIDO riguarda l'**autenticazione** (stabilire chi sei), non l'autorizzazione (cosa puoi fare). Confondere autenticazione/autorizzazione è l'errore più comune su S2 — li hai distinti bene nell'introduzione AAA, ma è scivolato qui.

FIDO UAF (Universal Authentication Framework): autenticazione *senza password*. L'utente si autentica localmente sul dispositivo (biometria o PIN); il dispositivo usa questa verifica per sbloccare una chiave privata che firma la sfida del server FIDO UAF. Le informazioni di sicurezza **non lasciano mai il dispositivo**.

FIDO U2F / FIDO2 / WebAuthn: rafforza e semplifica la 2FA usando dispositivi hardware (USB, NFC, Bluetooth). La **YubiKey** è l'implementazione più nota.

Flusso U2F: 1. Browser invia la sfida al dispositivo 2. L'utente tocca il pulsante fisico (prova di presenza umana) 3. Il dispositivo firma la sfida con la propria chiave privata 4. Il server verifica con la chiave pubblica associata

Anti-phishing integrato: la YubiKey esegue un *origin-check* — verifica il dominio del sito prima di firmare. Un sito fake (g00gle.com) riceve una firma valida solo per quel dominio, non usabile su google.com.

⚠ Questa sezione (YubiKey/WebUSB 2018) non era presente negli appunti grezzi.

Nel 2018, Vervier e Orrù hanno dimostrato che la feature **WebUSB** di Chrome (comunicazione diretta JS → USB) poteva essere usata per aggirare l'*origin-check* della YubiKey NEO, inviando richieste direttamente all'interfaccia CCID del dispositivo. Il punto non è che la YubiKey è insicura — è che la sicurezza dell'hardware dipende dall'**isolamento del contesto software** che lo circonda. La vulnerabilità stava in Chrome, non nella chiave.

Domande di autoverifica — Risposte

1. Username inserito → risposta: **C** — **Identificazione** ✓
2. Il salt protegge contro attacchi offline? → **Falso** ✓ Il salt impedisce le rainbow table, non il brute force diretto su /etc/shadow rubato. La protezione offline è l'entropia della password.
3. OTP via SMS è 2FA o 2SA? → **B** — **2SA** ✓ (SMS intercettabile via MITM, non è “possesso” affidabile)
4. Differenza *required* vs *requisite* in PAM → “non ho capito l'argomento”

Risposta: entrambi i flag implicano che il modulo *deve* passare perché lo stack riesca. La differenza è sul *quando* si rivela il fallimento: - *required*: il fallimento viene registrato internamente, ma lo stack **continua** verso i moduli successivi (così chi attacca non sa quale modulo ha rifiutato). Lo stack restituisce fallimento alla fine. - *requisite*: il fallimento **ferma immediatamente** lo stack con errore. Più veloce, ma rivela che c'è stato un blocco precoce. Usa *required* quando vuoi mascherare quale modulo ha fallito (sicurezza contro timing attack); usa *requisite* quando un certo fallimento rende inutile continuare (es. assenza di smart card).

5. S-KEY: chi intercetta $h^{(k-1)}(N)$ può riusarlo? → **Falso** ✓ (ma “non ho capito l'argomento”)

Risposta: vedi sezione S-KEY sopra. Il token intercettato è già consumato (V ha aggiornato il suo riferimento) e il successivo non è calcolabile perché richiederebbe invertire l'hash.

6. YubiKey + WebUSB: la YubiKey protegge contro qualsiasi attacco? → **Falso**  La vulnerabilità WebUSB 2018 ha dimostrato che un contesto software compromesso (Chrome) può aggirare l'origin-check.

Risultato autoverifica: 4/6 risposte corrette al primo tentativo. Q4 e Q5 non erano chiare — entrambe le risposte sono ora integrate in questa nota.

Collegati

- [\[\[lezione_moduloS2_autenticazione\]\]](#)

Hub: [\[\[master_map_studio\]\]](#) · [\[\[concept_maps\]\]](#) · [\[\[metodo_studio_esami_pratici\]\]](#)